Questa è una copia di una conversazione tra ChatGPT e un utente anonimo.
Segnala la conversazione

Quiero hacerte una consulta (en otro tab estoy trabajando en un nuevo desarrollo) para la elaboración de un exchange y self-custoded wallet que sea de aspecto y/o funcionalidades a la de Binance (con features "básicas estándares".
ME redactarías de forma correcta cómo deberiá de pasarte la información para que puedas proceder con el

ChatGPT Accedi Registrati gratuitamente

Perfecto, esto que querés hacer (exchange + wallet self-custody tipo Binance) es **complejo pero totalmente abordable si lo estructuramos bien desde el inicio**.
La clave no es que me pases “ideas sueltas”, sino un **brief técnico estructurado** para poder diseñarlo correctamente (arquitectura + código + roadmap).

Te dejo **exactamente cómo deberías pasarme la info** (copiar/pegar y completar 👇).
Esto está pensado para que yo después pueda devolverte directamente: arquitectura, DB, APIs, y base de código.

📦 TEMPLATE – ESPECIFICACIÓN DEL PROYECTO (EXCHANGE + WALLET)

1. 🧠 VISIÓN GENERAL

- Nombre del proyecto:
- Objetivo principal (ej: DEX, CEX, híbrido):
- Público objetivo:
- Países donde operará:
- Nivel de compliance requerido:
  - ☐ MVP sin regulación
  - ☐ Preparado para regulación (KYC/AML)
  - ☐ Full regulado

---

2. 🏗️ TIPO DE PLATAFORMA

Elegir:

- ☐ Centralized Exchange (CEX)
- ☐ Decentralized Exchange (DEX)
- ☐ Híbrido (recomendado)

Aclarar:

- ¿Custodia de fondos?
- ¿Self-custody real (wallet del usuario)?
- ¿Habrá cuentas internas tipo Binance?

---

3. 💰 FUNCIONALIDADES CORE

Marcar lo que querés:

Trading

- ☐ Spot trading
- ☐ P2P
- ☐ Futures (más adelante)
- ☐ Margin trading

Wallet

- ☐ Wallet custodial
- ☐ Wallet self-custody
- ☐ Importar wallet (seed / private key)
- ☐ Generar wallet nueva
- ☐ Multi-chain (cuáles?)

Assets

- Criptos iniciales:
- Tokens propios (ej: SCAVO token):
- ¿ERC20 / BEP20 / nativo?

---

4. 🌐 BLOCKCHAIN / INFRAESTRUCTURA

- Redes:
  - ☐ Ethereum
  - ☐ BSC
  - ☐ Polygon
  - ☐ Otra: _____
- Nodo propio:

    - ☐ Sí
    - ☐ No (usar RPC externo)
- Custodia:
    - ☐ Hot wallet
    - ☐ Cold wallet
    - ☐ MPC (opcional futuro)

---

5. 🔐 SEGURIDAD

- ☐ 2FA
- ☐ Email verification
- ☐ Anti-phishing
- ☐ Withdrawal whitelist
- ☐ Rate limiting
- ☐ Firma de transacciones local (wallet)

---

6. 👤 USUARIOS

- Registro:
    - ☐ Email
    - ☐ Teléfono
    - ☐ Wallet login (Web3)
- KYC:
    - ☐ No
    - ☐ Básico
    - ☐ Completo

---

7. 💻 FRONTEND

Preferencias (sé que usás Flutter):

- ☐ Flutter Web
- ☐ Flutter Mobile
- ☐ Desktop

Estilo:

- ☐ Tipo Binance
- ☐ Minimalista
- ☐ TradingView integrado

---

8. ⚙️ BACKEND (IMPORTANTE)

(Sé que usás Go, pero confirmame)

- Lenguaje:
- WebSockets:
- Base de datos:
- Cache:
- Cola de tareas:

Ejemplo ideal:

- Go + WebSocket + MySQL + Redis

---

9. 📊 MOTOR DE TRADING (CLAVE)

Definir:

- ☐ Order book interno
- ☐ Matching engine propio
- ☐ Uso de liquidez externa (API Binance, etc.)

---

10. 💸 DEPÓSITOS / RETIROS

- Cripto:
  - ☐ Sí
- Fiat:
  - ☐ Sí / No
- Métodos fiat:
  - ☐ Transferencia bancaria
  - ☐ MercadoPago
  - ☐ Otros

---

11. 📡 INTEGRACIONES

- ☐ TradingView charts
- ☐ APIs externas (cuáles?)
- ☐ Notificaciones (email / push / WhatsApp)

---

12. 🚀 ETAPAS (MUY IMPORTANTE)

Cómo querés avanzar:

Ejemplo:

- Fase 1: Wallet + login
- Fase 2: Depósitos/retiros
- Fase 3: Trading básico
- Fase 4: Escalado

---

13. 🧩 COSAS ESPECÍFICAS TUYAS (MUY IMPORTANTE)

Acá meté TODO lo custom tuyo:

- Integración con tu ERP (Geryon)
- Sistema de clientes existente
- Sistema de billing
- Infraestructura actual (servers, Mikrotik, etc.)
- Tokens propios (SCAVO)

🚀 CÓMO VAMOS A TRABAJAR DESPUÉS

Una vez que me pases esto, yo te devuelvo:

1. 🧠 Arquitectura completa (tipo Binance-like realista)
2. 🗄️ Modelo de base de datos
3. 🔌 APIs (REST + WebSocket)
4. ⚙️ Matching engine (en Go, optimizado)
5. 💻 Base de frontend en Flutter
6. 🔐 Diseño de seguridad
7. 📦 Roadmap técnico por fases
8. Código inicial listo para correr

💡 RECOMENDACIÓN IMPORTANTE (DE EXPERIENCIA)

No intentes arrancar como Binance completo.

👉 Lo correcto:

1. Wallet + depósitos
2. Balance interno
3. Trading simple
4. Escalar

Si querés, en el próximo paso:
👉 completás esto
👉 y yo te armo directamente la **arquitectura completa + stack exacto adaptado a tu ecosistema (Go + Flutter + tu ERP)**

Y lo dejamos listo para producción desde el día 1.

📦 TEMPLATE – ESPECIFICACIÓN DEL PROYECTO (EXCHANGE + WALLET)
1. 🧠 VISIÓN GENERAL

Nombre del proyecto: SCAVO Exchange

Objetivo principal (ej: DEX, CEX, híbrido): híbrido (nos enfocamos en DEX)

Público objetivo: usuario final (masivo)

Países donde operará: Argentina

Nivel de compliance requerido: MVP sin regulación (luego vamos avanzando conforme vamos probando)

MVP sin regulación

Preparado para regulación (KYC/AML)

Full regulado
2. 🏗️ TIPO DE PLATAFORMA Híbrido (recomendado) < inicial: DEX

Elegir:

Centralized Exchange (CEX)

Decentralized Exchange (DEX)

Híbrido (recomendado)
* Self-custody real (wallet del usuario)
* Cuentas internas tipo Binance

3. 💰 FUNCIONALIDADES CORE

Marcar lo que querés:

Trading <no ahora>

Spot trading <si>

P2P <si - prioridad>

Futures <más adelante>

Margin trading <no ahora>

Wallet

Wallet custodial <si>

Wallet self-custody <si>

Importar wallet (seed / private key) <si>

Generar wallet nueva <si>

Multi-chain (cuáles?) <si> - BSC, Ethereum y SCAVIM (full ethereum)
Assets

Criptos iniciales: BSC, ETH, SCAVO (nativo)

Tokens propios (ej: SCAVO token): SCAVO Token

¿ERC20 / BEP20 / nativo?: ERC20, BEP20 y nativo (SCAVO)

4. 🌐 BLOCKCHAIN / INFRAESTRUCTURA

Redes:

Ethereum

BSC

Polygon

SCAVIUM

Nodo propio: Sí (para propio)

No (usar RPC externo) (para demás)

Custodia:
Hot wallet <si>

Cold wallet <no por ahora>

MPC (opcional futuro) <no sé que es>

5. 🔐 SEGURIDAD

2FA <si, 2da etapa>

Email verification <si>

Anti-phishing <no por ahora>

Withdrawal whitelist <si, opcional>

Rate limiting <si, opcional según tu sugerencia>

Firma de transacciones local (wallet) <si, opcional>

6. 👤 USUARIOS

Registro:

Email <si>

Teléfono <no, por ahora>

Wallet login (Web3) <si>

KYC: No

Básico <no ahora>

Completo <no ahora>

7. 💻 FRONTEND

Preferencias (sé que usás Flutter):

Flutter Web <si>

Flutter Mobile <si, segunda etapa>

Desktop <si>

Estilo:

Tipo Binance <si>

Minimalista<no>

TradingView integrado<no por ahora>

9. 📊 MOTOR DE TRADING (CLAVE)

Definir:

Order book interno <si>

Matching engine propio <si>

Uso de liquidez externa (API Binance, etc.) <no por ahora, sugerir mas adelante>

10. 💸 DEPÓSITOS / RETIROS

Cripto: Sí

Fiat: <no por ahora, podría dejarse una interfaz para simular y probar>

Métodos fiat:

Transferencia bancaria <si>

Otros <sugerencia>

11. 📡 INTEGRACIONES

TradingView charts <si se puede>

APIs externas (cuáles?) <no por ahora, sugerir>

Notificaciones (email / push / WhatsApp) <email, push, whasapp no por ahora>

12. 🚀 ETAPAS (MUY IMPORTANTE)

Cómo querés avanzar:

Ejemplo:

Fase 1: Wallet + login

Fase 2: Depósitos/retiros

Fase 3: Trading básico

Fase 4: Escalado

Perfecto. Con lo que completaste ya se puede **bajar a arquitectura + backlog técnico + estructura de repos** sin inventar demasiado.

A continuación te devuelvo:

1. **Cómo interpreto tu especificación** (para evitar malos entendidos)
2. **Arquitectura propuesta (MVP → etapas)**
3. **Modelo de dominios / componentes clave**
4. **Backlog por fases (con entregables claros)**
5. **Decisiones que te conviene tomar ahora (sin frenar el avance)**
6. Qué me tenés que pasar *sí o sí* para arrancar a codear (sin preguntas "declarativas", cosas concretas)

---

1) Interpretación de tu spec (lo que queda definido)

- Proyecto: **SCAVO Exchange**
- Tipo: **Híbrido**, pero el **MVP inicial se enfoca en DEX**.
- Usuario final masivo en **Argentina**.

- Compliance: **MVP sin regulación**, pero dejando “ganchos” para KYC/AML después.
- Wallet: **custodial + self-custody real** (ambas coexistiendo).
- Multi-chain:
  - **SCAVIUM** (tu chain, EVM full Ethereum) con **nodo propio**
  - **Ethereum** y **BSC** con RPC externos (por ahora)
- Trading:
  - Spot: sí (no inmediato)
  - P2P: sí (**prioridad**)
  - Engine: **order book interno + matching propio**
- Depósitos/retiros: cripto sí; fiat no en MVP, pero interfaz “simulada”
- Frontend: Flutter Web sí; Mobile/desktop etapa 2
- Seguridad: email verification sí; 2FA etapa 2; rate limit recomendado; whitelist opcional

---

2) Arquitectura propuesta (realista y escalable)

Capas (alto nivel)

A) Apps

- Flutter Web (MVP)
- Flutter Mobile/Desktop (etapa 2)

B) Backend (Go)

- API Gateway (REST)
- WS Gateway (WebSocket para eventos)
- Auth Service (email + Web3 login)
- Wallet Service (custodial + self-custody helpers)
- Chain Service (indexación + broadcast tx + confirmaciones)
- Ledger Service (balances internos + movimientos)
- P2P Service (escrow + disputas simples)
- Trading Engine (más adelante / fase 3)
- Notification Service (email; luego push)

C) Infra

- DB: MySQL o Postgres (si estás muy casado con MySQL, ok)
- Redis: sesiones, rate-limit, colas livianas, locks
- Queue: al principio Redis streams o tu sistema de workers (vos ya tenés)
- Observabilidad: logs estructurados + métricas

Concepto clave (para que “híbrido” no explote)

Vas a tener **dos mundos de saldo**:

1. **Self-custody**: saldo vive en la blockchain (metamask-like).
   - Vos solo indexás, mostrás y ayudás a firmar/broadcast.
2. **Custodial (tipo Binance)**: saldo vive en tu **ledger interno**, con hot wallets para depósitos/retiros.
   - Esto habilita P2P fluido + trading rápido.

Esto te permite arrancar DEX/self-custody sin tener que construir todo el core de custodia, *pero* igual construir custodial "acotado" para P2P y luego trading.

---

3) Componentes clave (dominios que vamos a modelar)

Identidades y acceso

- Usuario (email) + verificación
- Wallet login (firma de mensaje EIP-4361 tipo "Sign-In With Ethereum" o equivalente EVM)
- Sesiones / refresh tokens
- Roles (admin/support) aunque sea básico

Wallet / Chain

- Address book por usuario
- Import/generate wallet:
  - Opción A (recomendada): **self-custody siempre en el cliente** (seed nunca toca el server)
  - Opción B (custodial): el server crea y guarda keys (más riesgo, pero habilita UX)
- Indexación:
  - Detectar depósitos
  - Confirmaciones
  - Reorg handling (EVM)

Ledger interno (para custodial)

- Accounts (por usuario, por asset, por "wallet-type")
- Movimientos atómicos (doble partida ideal)
- Reservas (hold) para P2P/trading

P2P (tu prioridad)

- Offer (publicación)
- Order (toma de oferta)
- Escrow (hold del balance custodial)
- Estados: created → locked → paid → released / cancelled / disputed

- Evidencias (comprobantes) opcional
- Disputas manuales (admin)

Trading engine (fase 3)

- OrderBook por par
- Orders: limit/market (arrancamos con limit)
- Matching
- Trades
- Fees

---

4) Roadmap por fases (con entregables)

Fase 0 — Base técnica (fundación)

Objetivo: repo corriendo, auth, DB, CI mínimo.

- Repo layout (Go monorepo o multi)
- Config/secret management
- Migraciones DB
- Logger + tracing básico
- Healthchecks

Entregable: `docker-compose up` y tenés API viva.

Fase 1 — Login + Wallet self-custody + Explorer interno

Objetivo: "Me logueo, conecto wallet, veo balances on-chain".

- Registro email + verificación
- Login por Web3 (firma de challenge)
- Selector de chain (ETH/BSC/SCAVIUM)
- Indexador mínimo (leer balances ERC20 / nativo)
- UI Flutter: Home + Wallets + Assets

Entregable: MVP usable para self-custody sin custodial.

Fase 2 — Custodial mínimo + Depósitos/Retiros cripto (Hot wallet)

Objetivo: "Tengo balance interno y puedo mover cripto".

- Ledger interno (balances + movimientos)
- Depósitos: generar address por usuario (o memo) y reconocer

- Retiros: request → revisión simple → broadcast → confirm
- Whitelist opcional
- Rate limit / antifraude básico

Entregable: base “tipo exchange” con custodia, pero sin trading aún.

Fase 3 — P2P (prioridad tuya)

Objetivo: “Compro/vendo P2P con escrow interno”.

- Offers + Orders + Escrow (hold)
- Estados y timeouts
- Chat simple opcional (o solo comentarios)
- Admin panel mínimo para disputas
- Notificaciones email

Entregable: producto con tracción real sin motor de trading.

Fase 4 — Spot trading (order book + matching)

Objetivo: “Spot simple tipo Binance Lite”.

- Mercado por pares (SCAVO/USDT, ETH/USDT, etc.)
- Órdenes limit (luego market)
- Matching engine
- WebSocket events (ticker/orderbook/trades/user fills)
- Fees

Entregable: Spot funcional.

---

5) MPC: qué es y por qué importa (en simple)

MPC (Multi-Party Computation) es una forma de custodia donde **no existe una sola “private key completa” en un lugar**. La firma se hace entre varias “partes” (servidores/dispositivos), reduciendo el riesgo de robo de una key única.
No hace falta para MVP, pero es una mejora fuerte para custodial en serio.

---

6) Lo único que necesito ahora para arrancar desarrollo (checklist)

Sin esto, se puede avanzar igual, pero esto acelera muchísimo:

A) Definiciones de activos/pairs iniciales

- Lista de tokens iniciales por red (contract addresses en ETH/BSC; y en SCAVIUM si aplica)
- Qué vas a usar como “moneda base” para P2P:
  - USDT? USDC? ARS tokenizado? (aunque sea simulado)

B) SCAVIUM

- RPC endpoint (interno) + chainId + explorer si existe
- Cómo emitís SCAVO nativo: ¿es coin nativa de SCAVIUM o token ERC20 dentro de SCAVIUM?

C) Custodial key management (decisión rápida)

Para MVP te recomiendo:

- **Self-custody:** claves solo en el cliente (Flutter)
- **Custodial:** hot wallet del server con llave en vault/secret (mínimo), con rotación más adelante

Confirmame si aceptamos esto como estándar.

D) UX “Binance-like” que querés copiar primero

Elegí 3 pantallas para arrancar:

- Home
- Wallet
- P2P
- Spot
- Profile/Security

Con elegir el orden, ya seguimos.

Próximo paso (ya operativo)

Si me decís:

1. **stack backend** (Go + MySQL/Redis confirmado?)
2. **assets iniciales** (aunque sea 3 tokens + 1 stable)
3. datos mínimos de **SCAVIUM** (RPC + chainId)

yo te devuelvo en el próximo mensaje:

- **estructura de repos** (carpetas, módulos, naming)
- **modelo de DB inicial (migraciones)** para auth + wallets + ledger + p2p
- **API spec** (REST + WS)
- primer **sprint de implementación** con código base Go + Flutter

Y arrancamos con Fase 0/Fase 1 inmediatamente.